🚨 HikerNet Trek Module: Deep-Dive Error & Vulnerability Report

While the Trekking Module functions as the core of the HikerNet ecosystem, several underlying structural, logic, and hardware-dependent issues hinder its stability. This report highlights the existing errors, specifically focusing on pathing inaccuracies, backtracking logic, and state desynchronizations.

---

1. 🧭 The "Track Back" / Overlapping Path Issue

One of the most profound issues currently affecting hikers in out-and-back trails (where the return journey uses the exact same path as the outbound journey) is the breakdown of the **GeoJSON LineString** rendering and distance calculations.

Technical Breakdown:

- **The Issue**: When a user reaches a summit and turns around to "trek back", the GPS coordinates being passed to the backend begin overlapping with the outbound coordinates.
- **Distance Calculation Errors**: Because the algorithm relies on raw point-to-point Haversine distance computations (`getDistance`), minor GPS drifts on the return journey can cause the system to hyper-calculate zig-zags across the original path. A 5km out-and-back trail can incorrectly log as 13km due to micro-variations in the overlapping coordinate arrays.
- **Rendering Artifacts**: On the `/explore.jsx` React Native MapLibre wrapper, overlapping `[longitude, latitude]` arrays often cause rendering engine confusion. The drawn vector line can glitch, creating visual geometric "spiderwebs" rather than a clean overlapped line.
- **The Missing "Return" State**: The schema (`models/Trek.js`) does not distinguish between outbound waypoints and return waypoints. There is no built-in `isBacktracking` flag, making it impossible for the frontend to differentiate the rendering layers.

---

2. 🛰️ GPS Signal & Sensor Fusion Failures (`useSmartLocation.js`)

While mitigation has been applied via `useSmartLocation` , the reliance on device-specific hardware exposes Several edge cases.

The Pedometer Gating Flaw

- **The Issue**: To prevent "Stationary Drift" (GPS wandering when standing still), the code relies on `expo-sensors` Pedometer. If the pedometer reports `isWalking = false` , GPS updates `< 0.5 meters` are rejected.
- **The Flaw**: If a user is mountain biking, rock climbing (vertical ascent with minimal step impact), or riding in an emergency vehicle off the mountain, the pedometer will register `0 steps` . Consequently, the app will falsely assume the user is "Stationary" and fiercely reject genuine GPS updates, freezing the trek path entirely.

Accuracy Floor Cutoffs

- **The Issue**: The script currently rejects any GPS coordinate with an accuracy rating worse than `100 meters` .
- **The Flaw**: In dense evergreen forests, deep canyons (slot canyons), or heavy cloud cover, real GPS accuracy naturally drops to 150m+. Instead of plotting a rough path, the app throws a `REJECT:`

`Poor Accuracy` console log. The user might hike 3 miles through a canyon and returning zero tracked distance.

---

3. 👥 Group Trek State Desynchronization

The `Room.js` and group syncing architecture suffers from distinct race conditions and polling overhead.

Silent Disconnects

- **The Issue**: When trekking in a group, if a member enters a true cellular dead zone, their local state fails to hit the `/api/treks/:id/waypoint` endpoint.
- **The Flaw**: The group tracking logic does not currently handle queueing or background sync efficiently. When the user regains signal, they do not bulk-upload their missed coordinates. Instead, they simply emit their *current* location. This results in straight-line teleportation on the map from where they lost signal to where they regained it, slicing through impassible terrain visually.

Photo & Waypoint Duplication

- **The Issue**: In a shared Group Trek session, if two users standing at the same viewpoint attempt to add a Waypoint with an image simultaneously, the backend lacks a robust idempotency key.
- **The Flaw**: The `/api/treks/:id/waypoint` endpoint will append two identical geospatial waypoints to the array. Furthermore, the Cloudinary image uploads could process out-of-sync, leading to duplicated UI renders in the post-trek review screen.

---

4. 🔋 Background Execution & OS Throttling

A fundamental flaw in the React Native / Expo implementation is the reliance on the active JavaScript thread for vital location polling.

- **The Issue**: iOS and Android aggressively suspend JavaScript execution when the app is backgrounded or the screen is locked to save battery.
- **The Flaw**: While `expo-location` has background capabilities natively, the current architecture pools locations in React `useState` and relies on `useEffect` timers to chunk HTTP payloads. When the screen locks, these states freeze. Users must keep their screens powered on (draining battery rapidly) to ensure accurate, high-fidelity tracking. If a user pockets their locked phone for 2 hours, the app may only register 5 data points.

---

5. 📉 Elevation Logic Hallucinations

- **The Issue**: Total Elevation Gain is a crucial metric for hikers, yet it is currently highly inaccurate.
- **The Flaw**: The altitude returned by raw GPS is notoriously imprecise (often fluctuating +/- 30 meters standing still). The backend simply calculates total elevation gain by adding the positive deltas between `point[n]` and `point[n-1]`. Because of standard GPS altitude noise, a flat walk on the beach can register as 500 meters of cumulative elevation gain, ruining the accuracy of the gamified Leaderboards.

🛠️ Summary of Required Architectural Fixes

1. **Implement Track-Back Logic**: Introduce directional vectors and path smoothing (e.g., Douglas-Peucker algorithm) to filter out raw GPS noise on overlapping paths.

2. **Move to Native Background Tasks**: Migrate the chunked payload uploader out of the React lifecycle and into `expo-task-manager` headless background execution.
3. **Queue-Based Offline Sync**: Build local SQLite/AsyncStorage queues that hold waypoints and telemetry when offline, bulk-syncing with distinct timestamps upon reconnection.
4. **Altimeter/Barometer Preference**: Rewrite elevation logic to prefer device Barometer pressure changes over raw GPS altitude arrays to stop elevation drift.